

API Guide

Public entry point

Normal clients call only the Gateway:

```
http://localhost:8080
```

Main business prefix: /api/v1.

Anonymous Gateway routes:

- POST /api/v1/auth/register
- POST /api/v1/auth/registration/verify
- POST /api/v1/auth/login
- POST /api/v1/auth/refresh

Other /api/* routes require JWT at the Gateway. FinWallet.Api independently repeats JWT, authorization and ownership checks.

Common HTTP contract

All success/failure bodies use `ServiceResult<T>`. Client logic should branch on:

- 1 HTTP status;
- 2 stable code.

Human-readable message text must not be parsed for business logic.

Correlation

Clients may send X-Correlation-Id. The shared web platform accepts only bounded alphanumeric/-/_ values; invalid or absent values are replaced. Correlation IDs are not transaction or idempotency keys and must not contain PII.

Authentication

Protected endpoints use:

```
Authorization: Bearer <access-token>
```

JWT is validated at both Gateway and FinWallet.Api. Money-changing transfer also validates JWT sid against durable CustomerSession state.

Swagger

Development:

Gateway	http://localhost:8080/swagger
FinWallet.Api	http://localhost:8081/swagger
FakeBank.Api	http://localhost:8082/swagger
FakeFraud.Api	http://localhost:8083/swagger
FakeCutoff.Api	http://localhost:8084/swagger
FakeCampaign.Api	http://localhost:8085/swagger
FakeCommunication	http://localhost:8086/swagger

Swagger is disabled by default in production.

Authentication endpoints

****Register**** — POST /api/v1/auth/register Creates a pending customer and sends OTP through FakeCommunication. Success: HTTP 202.

****Verify**** — POST /api/v1/auth/registration/verify Verifies/consumes OTP and activates the customer.

****Login**** — POST /api/v1/auth/login Validates credentials and creates durable session, access token and refresh token.

****Refresh**** — POST /api/v1/auth/refresh Rotates an opaque single-use refresh token; reuse detection revokes the session family.

****Logout**** — not yet implemented as a public endpoint.

Wallet endpoints

****POST /api/v1/wallets**** — requires JWT.

```
{ "currency": "TRY" }
```

Supported currencies: TRY, USD, EUR. First create returns 201; repeating the same customer/currency returns the existing wallet with 200. New wallets start with zero available and blocked balance.

****GET /api/v1/wallets**** — lists wallets owned by the JWT subject.

Bank account

****POST /api/v1/bank-accounts**** — requires JWT.

```
{ "walletId": "<wallet-guid>" }
```

Flow:

```
Gateway JWT
-> API JWT + ownership
-> durable BankAccount(Opening)
-> SQL completes
-> Gateway /providers/bank/*
-> FakeBank
-> provider result validation
-> CAS state update
```

External HTTP never runs inside an open SQL transaction.

Wallet transfer

****POST /api/v1/transfers**** requires:

- JWT;
- active durable session;
- Idempotency-Key;
- owned/active source wallet;
- distinct active destination;
- same currency;
- positive/bounded amount;
- final fraud Allow;
- sufficient balance.

```
POST /api/v1/transfers
Authorization: Bearer <JWT>
Idempotency-Key: transfer-000001
Content-Type: application/json
```

```
{
  "sourceWalletId": "aaaaaaaa-1111-4111-8111-111111111111",
  "destinationWalletId": "bbbbbbbbb-2222-4222-8222-222222222222",
  "amount": 125.50
}
```

Order:

```
completed replay
-> session/risk
-> internal fraud
-> FakeFraud via Gateway
-> combined decision
-> atomic MSSQL posting
```

A completed replay returns the same immutable transaction without a second fraud evaluation or money movement.

Funding status

No public BankDeposit endpoint exists yet. Because a new wallet starts at zero, the source wallet cannot currently be funded after registration using public endpoints alone. Test fixtures must create a balanced FinancialTransaction + LedgerJournal/Entries rather than directly updating the wallet balance.

Internal/provider routes

```
/providers/bank/*
/providers/fraud/*
/providers/cutoff/*
/providers/campaign/*
/providers/communication/*
```

These are not public client APIs. Gateway requires internal-service authorization and destination business endpoints additionally validate the downstream service key.

Platform failure examples

- missing/invalid JWT: 401 GATEWAY_UNAUTHORIZED;
- direct backend without downstream key: 401 INTERNAL_SERVICE_UNAUTHORIZED;
- rate limit: 429 RATE_LIMITED;
- unsupported write content type: 415;
- TRACE/CONNECT: 405;
- body/header limits: rejected before business processing.